

ASIS 2025 CTF

(Sawit slayer)

Team Profile



Sawit slayer

Invitation Code:

#	NAME	EMAIL
1	 Y4E3	surya
2	 Tongel	henal

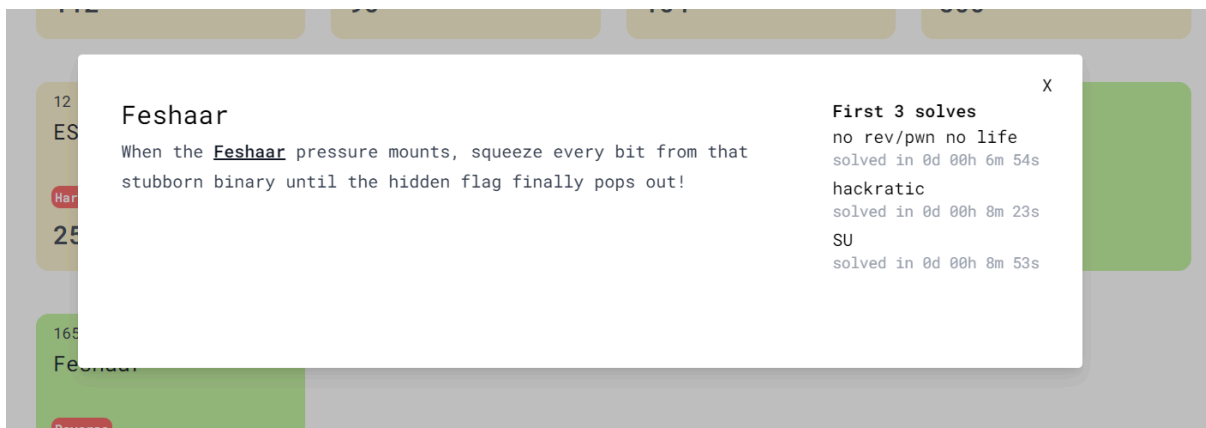
WELCOME



FLAG : ASIS{H4pPy_n3W_Y3Ar_&_7h3_B3st_of_Luck_1n_2026!}

REVESER

FESHAR



- First we are given a txz file
Feshaar_29ceb9ddca874fcb5dd90406b884afc3675e81c1.txz
- then we extract the file and in the folder there are files with the names feshar and flag.fshr
- Inside the feshar folder, there is an executable file named feshar and an output file named flag.fshr. The feshar file is the main program, while flag.fshr is the output file of the program. No original flag file was found in the folder, so it can be concluded that the flag has been processed or modified by the

program before being saved to flag.fshr. When the feshar program is run, no output is displayed in the terminal, and the program only produces the flag.fshr file, indicating that the program is working internally by processing a file.

- To understand the process, the feshar file was analyzed using Ghidra. The analysis revealed the main function,

```

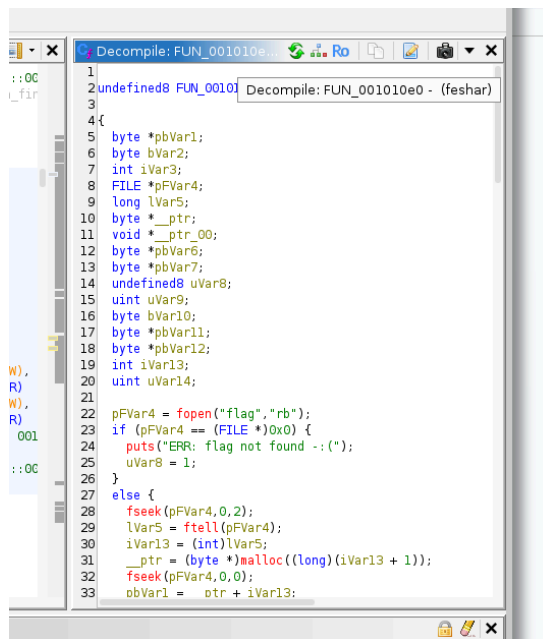
// SHT_PROGBITS [0x10e0 - 0x1478]
// ram:001010e0-ram:00101478
//

*****
*                               *
*                               *
*****
undefined8 __stdcall FUN_001010e0(void)
undefined8   RAX:8               <RETURN>
undefined1   Stack[-0x39]:1 local_39
undefined1   Stack[-0x3a]:1 local_3a
FUN_001010e0
XREF[2]:     0010128:
             001012a:
XREF[2]:     0010129:
             0010129:
XREF[4]:     entry:001013a4(*
             00102118(*),
             elfSectionHeade

```

FUN_001010e0, which opens the flag file, reads its entire contents, then compresses the data and stores it in flag.fshr. The program uses a simple two-byte compression algorithm: one control byte and one literal byte.

- Once the flag data is in memory, the program enters the core compression section.



- This section shows a loop comparing the current byte (pbVar11) with the previous bytes (pbVar7) to find a match. The check is performed sequentially up to a maximum of 7 bytes, as seen in the check sequence pbVar7[0] to pbVar7[6]. If the longest match is found, the length of the match is stored as uVar9, while the backward distance is calculated and stored in iVar3.

- The distance and length values are then combined into a single byte named bVar10 using the operation (byte)iVar3 | (byte)uVar9. The program then fetches the last literal byte (bVar2 = *pbVar12). These two bytes are always written in pairs to the output buffer __ptr_00. If the output buffer is full, its size is increased using realloc. This pattern indicates that the flag.fshr format always consists of two consecutive bytes: a control byte and a literal byte.
- Since the compression logic is clearly visible in Ghidra, the next step was to create a Python script to perform the reverse decompression of the algorithm. This script reads flag.fshr two bytes at a time, separates the control byte and the literal byte, then uses the bottom three bits as the data length and the remaining bits as the offset to recopy the old data.

```
def decompress_fshr(data: bytes) -> bytes:
    out = bytearray()
    i = 0

    while i < len(data):
        control = data[i]
        literal = data[i + 1]
        i += 2

        length = control & 0x07    # 3 bit bawah
        offset = control >> 3     # sisanya

        if offset != 0:
            start = len(out) - offset
            for j in range(length):
                out.append(out[start + j])

        out.append(literal)

    return bytes(out)

# === MAIN ===
with open("flag.fshr", "rb") as f:
    compressed = f.read()

plain = decompress_fshr(compressed)

# simpan hasil stage 1 (ASCII art)
with open("stage1.txt", "wb") as f:
    f.write(plain)

print("[+] Stage 1 decoded!")
print("[+] Output saved to stage1.txt")
```

- Cat stage1.txt

```
(Tongel@DESKTOP-IAC2P2K) ~/ASIS_25/Feshaar
$ cat stage1.txt
ASIS{C0mpr3s_Me_4Gain_&a9A!n}
```

FLAG : ASIS{C0mpr3s_Me_4Gain_&a9A!n}